

# Proposal: Schema-driven context-store binding (**x-context-source** + **x-context-target**)

**Status:** Draft for the platform / IDM team **Author:** vNext client SDK team **Scope:** Two backwards-compatible JSON-schema annotations so a generic client can wire a flow's inputs and persist its reusable outputs **without any per-flow client code** — purely from the backend schemas.

## 1. Problem

A generic client (our shell app boots from a single **clientId** and renders every flow from backend definitions) still has to **hardcode per-flow wiring**, because the schemas say *what* the fields are but never *where values come from* or *which outputs are worth keeping*:

- Which input fields are **user input** vs. **ambient** values the client already holds (channel, deviceId, installationId, clientId, the logged-in subject, ...). Today: hardcoded (device-token **customClaims**, the login **start** block).
- Which instance outputs are **reusable** and where they belong in the client's context-store (device id, certificate, tokens ...). Today: hardcoded token persistence in **completeLogin**, hardcoded device-registration storage.

Each new flow ⇒ new client code. That breaks "the client is generic; the backend drives everything."

## 2. Proposal — two mechanisms

Both are **optional** JSON-Schema annotations under a consistent **x-context-\*** namespace. JSON Schema (draft 2020-12) permits unknown keywords, so standard validators ignore them and a schema without them behaves exactly as today — **fully backwards compatible, adopt per-schema**.

| Annotation              | Lives on                                       | Direction                     | Applied when                                                 |
|-------------------------|------------------------------------------------|-------------------------------|--------------------------------------------------------------|
| <b>x-context-source</b> | a property of a <b>transition input schema</b> | context-store → input         | building a start/transition payload                          |
| <b>x-context-target</b> | the workflow's <b>master schema</b>            | instance data → context-store | on every instance read (start result, after each transition) |

### 2.1 **x-context-source** — read an input value from context-store

A property with an **x-context-source** is **resolved by the client** and not shown to the user. A property **without** one is user input (the client renders a form field, honoring **type** / **required** / **enum** / ...).

```
"properties": {
  "oldPassword": { "type": "string" },
  // no source → user input
  "channel":      { "type": "string", "x-context-source": { "const": "web" } }
```

```
  },
  "clientId":    { "type": "string", "x-context-source": { "const": "IbWeb" } },
},
"deviceId":    { "type": "string", "x-context-source": { "context": {
"boundary": "device", "key": "device.id" } } },
"userId":      { "type": "string", "x-context-source": { "identity":
"subject" } }      // logged-in subject (JWT sub)
}
```

2.2 x-context-target — write reusable instance data to context-store

On the **master schema** (the workflow-level data shape), **x-context-target** maps instance data fields → context-store locations. The client applies it **every time it reads the instance**, so values that appear only after a transition (deviceId, then the issued certificate, then tokens ...) propagate automatically and become available to later flows via **x-context-source**.

```
// device-manager master schema:
"x-context-target": {
  "deviceData.instanceId": { "context": { "boundary": "device", "key":
"device.registration.{instance}" } },
  "certificate":            { "context": { "boundary": "device", "key":
"device.cert.{instance}", "storage": "secure" } }
}
```

**{instance} templating.** The same logical field can arrive in different instances (e.g. one device-manager instance per device). Interpolating **{instance}** (the instance id/key) into the context-store key writes each instance's value to a distinct location instead of overwriting. Omit **{instance}** for cross-flow singletons that should live at a stable key (so a later flow's **x-context-source** can find them). Available template vars: **{instance}** (instance id), **{subject}** (active subject) — extensible.

3. Binding-location vocabulary

The value of an **x-context-source**, and the target of an **x-context-target** entry, is one of:

| Form                                                                                                             | Meaning                                                                                                   |
|------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| { "const": <any> }                                                                                               | A literal baked into the schema (source only).                                                            |
| { "context": { "boundary": "device user subject", "key": "<key-template>", "storage"?: "memory local secure" } } | A context-store <b>data</b> slot (key may contain {instance} / {subject}).                                |
| { "identity": "subject"   "user" }                                                                               | The context-store <b>identity</b> (activeSubject / activeUser), e.g. the logged-in userId. (Source only.) |

## 4. Client semantics (generic, no per-flow code)

**Building a start/transition payload** (reads the transition input schema):

1. Fetch the transition input schema (published schemas are queryable by key: `GET /api/v1/{domain}/workflows/sys-schemas/instances/{schemaKey}?sync=true` — **verified reachable**).
2. Partition **properties**: those with `x-context-source` are **resolved** (const / context / identity); the rest become **form fields**.
3. Submit **attributes** = **resolved** u **userInput**.

**Reading an instance** (start result, after each transition) — reads the master schema: 4. For each `x-context-target` entry: read the named field from the instance data; if present, write it to the resolved context-store location (interpolating `{instance}` = this instance's id). Reusable data now flows to context-store with zero flow-specific code, ready to be sourced by later flows.

Adding a new flow = publishing its schemas with the right annotations.

## 5. Why not a third "instance-key" annotation?

An earlier draft had an instance-key annotation for flows that resolve the subject from `context.Instance.Key` (e.g. `user-change-password`'s `get-user` reads `context.Instance?.Key`). **We dropped it** — `x-context-source` + `x-context-target` are the whole model. The instance-key case is better handled at the flow-design level:

**Recommendation:** flows should take identity as an `x-context-source` payload field (`"userId": { "x-context-source": { "identity": "subject" } }`) and read it from the payload, rather than requiring the caller to key the instance by the subject. Then plain `x-context-source` covers it and the generic client needs nothing extra.

If a flow must remain instance-keyed (legacy), that is the single case the generic client can't express with these two annotations alone — please flag such flows so we can discuss, rather than baking a third mechanism into the convention.

## 6. Worked examples

**update-password-input** (change-password start) — add an `x-context-sourced` `userId` (and let `get-user` read it from the payload instead of the instance key):

```
{
  "type": "object",
  "required": ["oldPassword", "newPassword", "newPasswordConfirm"],
  "properties": {
    "oldPassword": { "type": "string" },
    "newPassword": { "type": "string" },
    "newPasswordConfirm": { "type": "string" },
    "userId": { "type": "string", "x-context-source": {
      "identity": "subject" } } // NEW
  }
}
```

```
}
}
```

Client renders exactly the three password fields, injects `userId` from the logged-in subject, starts — **zero flow-specific client code**.

**user-login** master schema — persist issued tokens on read:

```
"x-context-target": {
  "accessToken": { "context": { "boundary": "user", "key":
    "auth.token.morph-idm-2fa.access", "storage": "memory" } },
  "refreshToken": { "context": { "boundary": "user", "key":
    "auth.token.morph-idm-2fa.refresh", "storage": "local" } }
}
```

Replaces the hardcoded persistence in `completeLogin`.

**device-token acquisition** — `channel/deviceId/installationId` become `x-context-source` payload fields instead of hardcoded `customClaims`.

## 7. Primary consumer: initialization workflows

The motivating use case for both annotations is **headless boot workflows**. Today the client **hardcodes** device registration (`device-manager`) and device-token acquisition to run on every launch — but different clients may not want them, and a client may need to run *different* setup flows at init (provision device, warm an SDK, fetch a config fact ...). These are almost always **non-visual** flows.

So `client-config` carries an ordered **initialization** array of workflows the client runs at boot (the field already exists — currently empty):

```
// client-config.initialization - ordered, headless boot workflows
"initialization": [
  {
    "key": "device-manager", "domain": "morph-idm", "version": "1.0.0",
    "mode": "headless", // run with no UI; drive auto-
    transitions to a settling state
    "optional": true, // tolerant: failure does not block
    boot (today's device behavior)
    "when": { "tokenLevel": "device" } // optional guard
  },
  { "key": "token", "domain": "morph-idm", "version": "1.0.0", "mode":
    "headless", "optional": true }
]
```

The client runs each entry **generically**, and this is exactly where the two annotations pay off — no per-flow code:

- **inputs** (deviceInfo, deviceId, installationId, channel, clientId, grantType ...) are resolved from the start schema's **x-context-source**;
- **outputs** (deviceId, certificate, **access\_token** ...) are persisted by the master schema's **x-context-target**, so later steps/flows read them back via **x-context-source**.

**tokenLevel** then follows from which tokens landed in context-store. This removes the hardcoded **provisionDevice** / **acquireDeviceToken** boot path entirely — device registration becomes just one declarative entry a client can keep, drop, or replace.

**initialization[]** itself is **client-config** (our shell side); the per-flow input/output binding is the **schema annotations** (platform/IDM side). The two compose into a fully backend-driven boot.

## 8. Context-store key registry (client ↔ schema contract)

Canonical locations the client exposes for schemas to reference; versioned with this convention:

| Reference                                                                                               | Value                                      |
|---------------------------------------------------------------------------------------------------------|--------------------------------------------|
| <code>{ "identity": "subject" }</code>                                                                  | logged-in userId (JWT <b>sub</b> )         |
| <code>context device / device.id</code>                                                                 | stable device id                           |
| <code>context device / installation.id</code>                                                           | per-session installation id                |
| <code>context device / device.registration.{instance}</code>                                            | device-manager registration (per instance) |
| <code>context device / device.cert.{instance}</code>                                                    | device certificate (per instance)          |
| <code>context device / auth.token.morph-idm-device.access</code>                                        | device token                               |
| <code>context user / auth.token.morph-idm-1fa.access</code>                                             | 1FA access token                           |
| <code>context user / auth.token.morph-idm-2fa.access</code><br>(memory) / <code>.refresh</code> (local) | 2FA tokens                                 |

## 9. Backwards compatibility & adoption

- Schemas **without** annotations behave exactly as today. Zero breakage.
- Adoption is **per-schema, incremental**.

## 10. Open question for the platform team

**Do unknown keywords (**x-context-source**, **x-context-target**) survive the schema publish + validation pipeline and come back intact on **GET .../sys-schemas/instances/{key}**?** We verified schemas are fetchable by key and return their **properties**, but we cannot deploy to the IDM environment to confirm custom keywords are preserved. If the pipeline strips unknown keywords, we would need either an agreed keyword allow-list or a single preserved **x-context-binding** container object.

## Appendix — client stopgaps to remove once adopted

- **Hardcoded boot path** `provisionDevice` + `acquireDeviceToken` (device registration + token run on every launch) → replaced by `client-config.initialization[]` (§7).
- **keyFrom:** `"activeUser"` + `startFields` in the shell nav config + `WorkflowRunner` (interim wiring for `user-change-password`) → replaced by `x-context-source`.
- Hardcoded device-token `customClaims` (channel / deviceId / installationId) → `x-context-source` payload fields.
- Hardcoded `user-login start` block (clientId / grantType / scopes) → `x-context-source`.
- Hardcoded token + device-registration persistence (`completeLogin`, `provisionDevice`) → `x-context-target`.